

CSE422: Artificial Intelligence
Summer 26

Project Report

Project Title: **Software Quality Prediction (High / Medium / Low)**

Section 20 ; **Group** 12

Nafi Nowaisir Chowdhury
ID: 24101162

Nowshed Alam Chowdhury Rahat
ID: 24101174

Table of Contents

1. Introduction 3
2. Dataset Description 3-7
3. Dataset Pre-processing 8-9
4. Dataset Splitting 9
5. Scaling and Normalization 9-10
6. Model Training & Testing (Supervised) 10-12
7. Model Selection and Comparison Analysis. 13-16
8. Conclusion 17-19

1. Introduction

Software quality assessment is a central concern in software engineering: identifying low-quality or defect-prone code early helps teams prioritize code review effort, reduce maintenance cost, and catch potential defects before they reach production. Manually auditing every module for quality is time-consuming and subjective, especially in large codebases with many contributors.

This project builds machine learning models that predict the quality label of a piece of software code - High, Medium, or Low - based on static code metrics such as lines of code, cyclomatic complexity, number of functions, code churn, comment density, number of bugs, presence of unit tests, and the experience level of the code owner. Software metrics like complexity and churn are well-known proxies for maintainability and bug-proneness in software engineering research; this project investigates how well quality can be predicted purely from these static, easily-collected code statistics, and where the limits of that prediction lie.

Since Quality Label takes three discrete category values (High, Medium, Low) rather than a continuous numeric value, this is a multi-class classification problem.

Project Link

2. Dataset Description

2.1 Dataset Overview

The dataset used in this project contains 1,600 synthetic records of software modules, each described by 8 code-level metrics and a 3-class quality label.

Property	Value
Number of Data Points	1,600 rows
Number of Input Features	8 input features + 1 target feature
Target Feature	Quality_Label (High / Medium / Low)
Feature Types	Mixed: Quantitative (numerical) and one binary categorical (Has_Unit_Tests)
Problem Type	Multi-class Classification (3 classes)
Class Balance	Close to balanced: High 35.4%, Low 33.3%, Medium 31.3%

Feature Descriptions

Feature Name	Type	Description
Lines_of_Code	Quantitative	Total lines of code in the module
Cyclomatic_Complexity	Quantitative	Number of independent paths through the code (complexity measure)
Num_Functions	Quantitative	Number of functions/methods defined in the module

Feature Name	Type	Description
Code_Churn	Quantitative	Amount of code change (lines added/modified/removed) over time
Comment_Density	Quantitative	Proportion of comment lines relative to code lines
Num_Bugs	Quantitative	Number of known bugs recorded for the module
Has_Unit_Tests	Categorical (binary)	Whether the module has unit tests (Yes/No)
Code_Owner_Experience	Quantitative	Years of experience of the module's primary owner
Quality_Label (Target)	Categorical (3 classes)	Overall quality rating: High, Medium, or Low

Why Multi-class Classification?

The target variable Quality Label takes three possible discrete values High, Medium, and Low rather than a continuous numerical output. Because there are more than two classes, this is specifically a multi-class classification problem rather than a binary one.

Encoding Requirement

Two columns required encoding before modeling. Has_Unit_Tests is a two-level category (Yes/No) and was mapped directly to 0/1 (binary encoding). Quality_Label is a three-level category and was converted to integers (0, 1, 2) using scikit-learn's LabelEncoder so it could serve as a numeric target for all models.

2.2 Correlation Analysis

A correlation heatmap was generated using the seaborn library to visualize linear relationships between all numerical/encoded features and the encoded target (Quality_Label_enc).

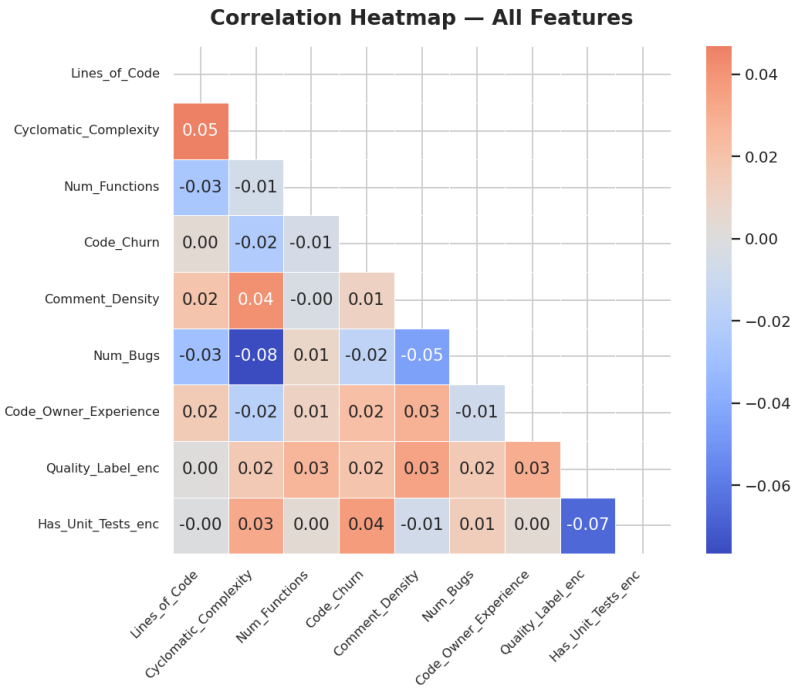


Figure 1. Correlation heatmap of all features, including the encoded target.

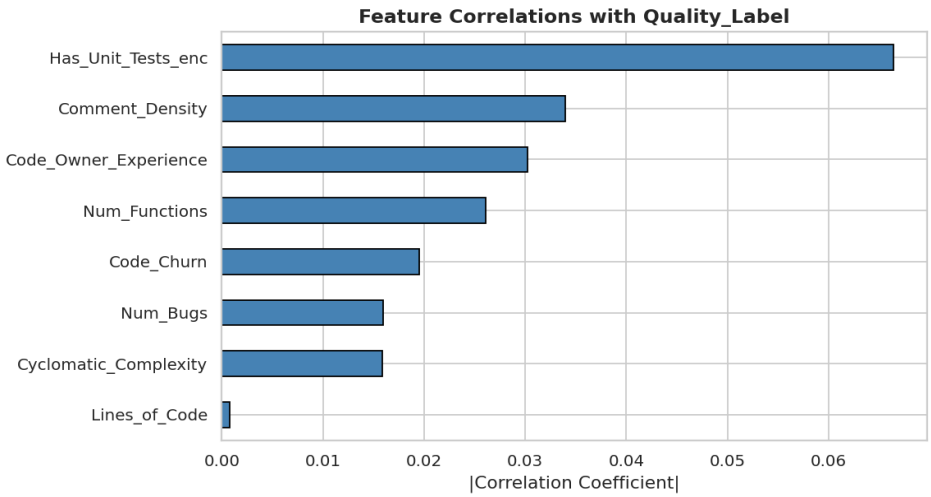


Figure 2. Absolute correlation of each feature with Quality_Label_enc, ranked.

The measured correlations between each feature and the target are:

Feature	**|Correlation| with Quality_Label**
Has_Unit_Tests	0.066
Comment_Density	0.034
Code_Owner_Experience	0.030
Num_Functions	0.026
Code_Churn	0.019

Feature	|Correlation| with Quality_Label
Num_Bugs	0.016
Cyclomatic_Complexity	0.016
Lines_of_Code	0.001

Every feature shows a very weak linear correlation with the target — the strongest, Has_Unit_Tests, is only 0.066. This is an important finding on its own: it indicates that the 8 available features carry little linear signal about Quality_Label, which foreshadows the low model accuracy reported in Section 6. No features were dropped for multicollinearity, since none of the pairwise correlations among the input features were strong enough to raise concern.

2.3 Class Balance Analysis

The distribution of the target variable Quality_Label was analyzed to check for class imbalance.

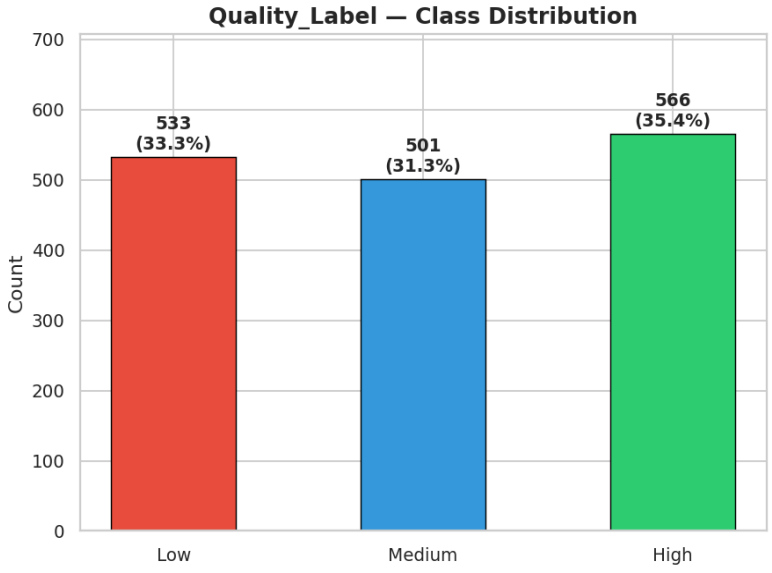


Figure 3. Class distribution of Quality_Label.

Class Label	Count	Percentage
High	566	35.4%
Low	533	33.3%
Medium	501	31.3%

The three classes are close in size (roughly 31–35% each), so the dataset is only mildly imbalanced at most. Aggressive resampling (e.g., SMOTE) was not required, though stratified splitting was still used to be safe and to preserve exact class ratios between train and test sets.

2.4 Exploratory Data Analysis (EDA)

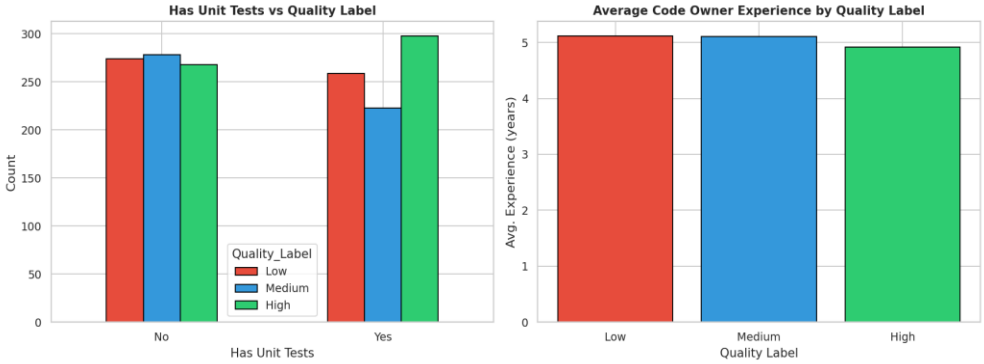


Figure 4. Left: Has_Unit_Tests vs Quality_Label counts. Right: average Code_Owner_Experience by Quality_Label.

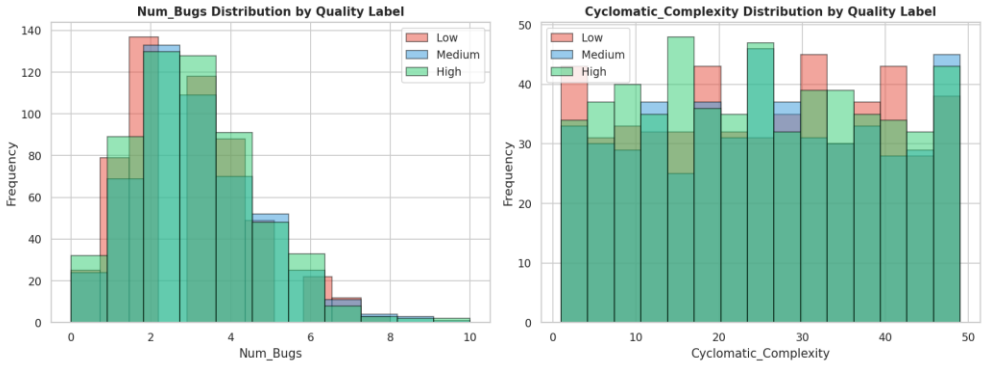


Figure 5. Left: Num_Bugs distribution by class. Right: Cyclomatic_Complexity distribution by class.

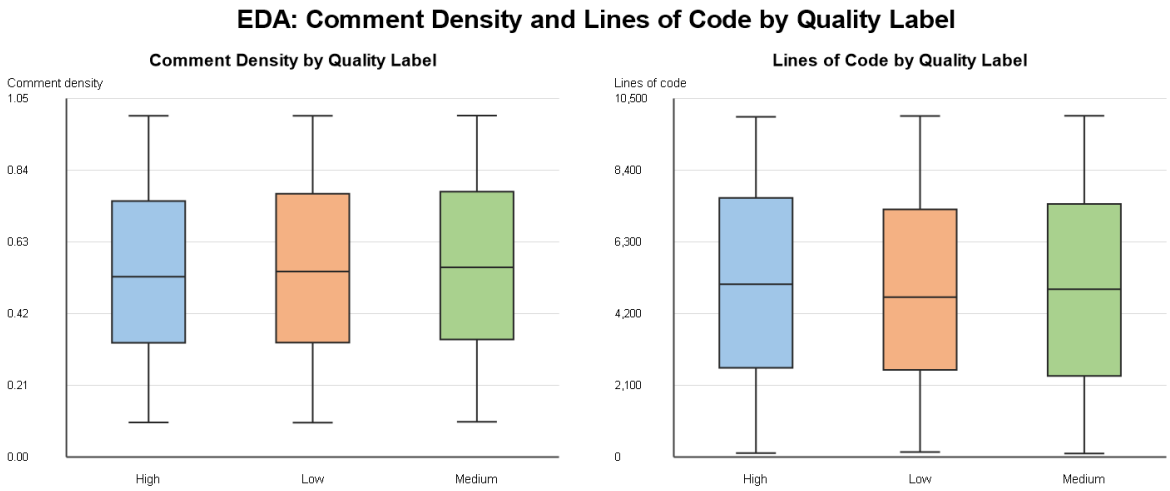


Figure 5a. Boxplots of Comment_Density and Lines_of_Code by Quality_Label.

Key observations from the EDA visualizations:

- Codebases with unit tests do not show a dramatically different quality-label mix than those without the bars across Low/Medium/High are broadly similar for both Has_Unit_Tests groups, consistent with the very low correlation (0.066) measured above.
- Average Code_Owner_Experience is nearly flat across Low, Medium, and High quality labels, rather than rising monotonically as might be expected more experienced owners are not clearly associated with higher-quality code in this dataset.
- Num_Bugs and Cyclomatic_Complexity histograms overlap heavily across the three quality classes; there is no clear separation where, for example, all Low-quality modules have visibly higher bug counts than High-quality ones.
- Comment_Density and Lines_of_Code also show highly overlapping ranges across High, Low, and Medium quality labels. The medians and interquartile ranges do not separate cleanly, so neither feature alone is a strong visual predictor of the target class.
- Overall, the EDA confirms what the correlation analysis suggested: none of the 8 features individually separates the three quality classes well.

3. Dataset Pre-processing

The pre-processing stage identified and addressed three main issues in the dataset. Each issue and its corresponding solution are described below.

Fault 1: Missing Values

Problem: Three numerical feature columns - Lines_of_Code, Code_Churn, and Comment_Density - each contained 80 missing values (5% of the 1,600 rows).

Solution: Median Imputation

- Median imputation (scikit-learn's SimpleImputer with strategy='median') was applied to all three columns rather than dropping the affected rows.
- Median is robust to outliers and skewed distributions, unlike the mean.

- Dropping the ~5% of affected rows would have discarded usable data in the other 5–8 features for those rows, at no real benefit.

After imputation, the dataset had zero remaining null values, confirmed programmatically, with the original 1,600-row sample size preserved.

Fault 2: Categorical Feature Encoding (Has_Unit_Tests)

Problem: Has_Unit_Tests is stored as text (Yes/No). Machine learning models require numeric input.

Solution: Binary Encoding

- Since Has_Unit_Tests is a two-level category, it was mapped directly: No → 0, Yes → 1.
- Binary mapping was preferred over One-Hot Encoding here because a single 0/1 column already captures the full information content of a two-level variable without adding redundant columns.

Fault 3: Target Column Encoding (Quality_Label)

Problem: The Quality_Label column contains string labels (High, Medium, Low). Models require numeric targets.

Solution: Label Encoding

- scikit-learn's LabelEncoder converted the three string labels to integers: High → 0, Low → 1, Medium → 2 (alphabetical order, as assigned by LabelEncoder).
- Quality_Label does carry a natural order (Low < Medium < High); label encoding does not preserve this ordinality, but all four models used here (Naive Bayes, Logistic Regression, Neural Network, and KMeans, via majority-vote mapping) treat the three classes as unordered categories rather than relying on the numeric ordering, so this is an acceptable simplification for this project.

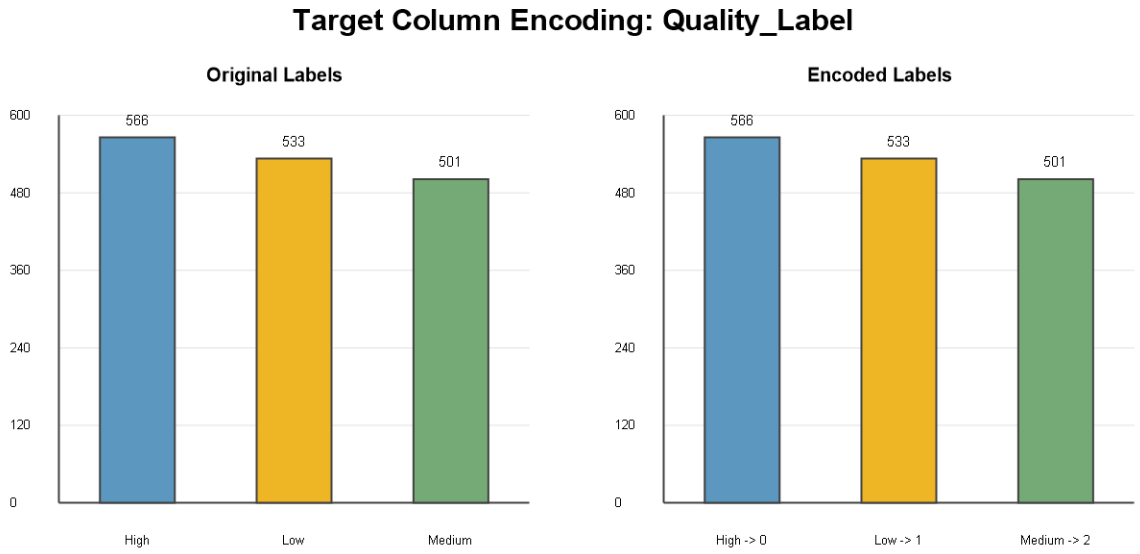


Figure 5b. Target column encoding for Quality_Label, showing original class counts and LabelEncoder numeric codes.

The target encoding graph confirms that the original labels remain close to balanced after encoding: High is encoded as 0, Low as 1, and Medium as 2. Encoding changes only the representation used by the models; it does not change the number of samples in any target class.

4. Dataset Splitting

The dataset was split using stratified splitting to preserve the original class distribution in both the training and test subsets.

Parameter	Value	Justification
Split Type	Stratified	Maintains the ~35% / ~33% / ~31% class proportions in both sets
Train Set	80% (1,280 samples)	Sufficient data for all models to learn from
Test Set	20% (320 samples)	Reasonable holdout size for reliable evaluation across all metrics
Random State	42	Ensures reproducibility across runs

Feature scaling (StandardScaler) was applied after splitting — fit on the training data only — to strictly prevent data leakage from the test set into the scaling parameters.

5. Scaling and Normalization

After the stratified train/test split, StandardScaler was applied to the feature matrix. The scaler was fit only on X_train and then used to transform both X_train and X_test, which prevents information from the test set from leaking into the training process.

Scaling was necessary because the features use very different numeric ranges: Lines_of_Code spans roughly hundreds to ten thousand, while Num_Bugs ranges from 0 to 10 and Comment_Density ranges from 0 to 1. Standardization converts the features to zero-centered, unit-variance values so Logistic Regression, the Neural Network, and K-Means are not dominated by the largest-scale columns.

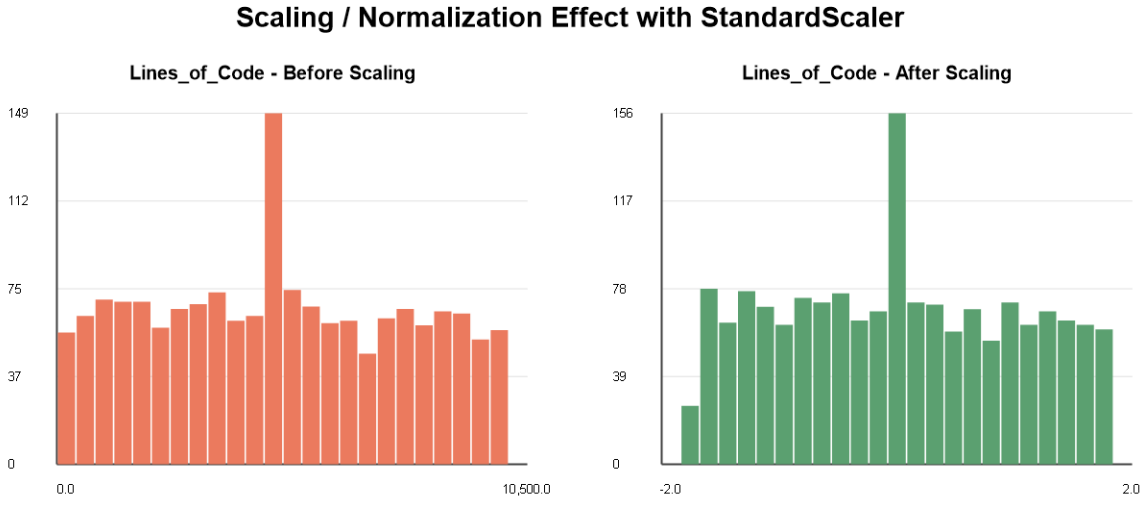


Figure 5c. Lines_of_Code distribution before and after StandardScaler normalization.

6. Model Training & Testing

Given that the classes are close to balanced, no class-weighting scheme was required. All four models below were trained on the same 8-feature, scaled training set and evaluated on the same held-out 20% test set.

6.1 Naive Bayes

Gaussian Naive Bayes is a probabilistic classifier based on Bayes' theorem, assuming conditional independence between features given the class label. It was implemented using scikit-learn's GaussianNB with default parameters.

Configuration	Details
Library	sklearn.naive_bayes.GaussianNB
Variant	Gaussian — appropriate since the numerical features are approximately continuous
Scaling Required	Not strictly required for Naive Bayes, but applied for consistency across models
Key limitation	Assumes features are conditionally independent given the class, which may not hold exactly for correlated code metrics

6.2 Logistic Regression

Logistic Regression is a linear classifier that models class-membership probability using the softmax function over three classes. It serves as an interpretable linear baseline. It was implemented with scikit-learn's LogisticRegression (solver='lbfgs', max_iter=1000, C=1.0), which handles multinomial classification natively.

- This model is sensitive to feature scale, so StandardScaler was essential before training.
- It assumes an approximately linear relationship between the (scaled) features and the log-odds of each class, which may be a limiting assumption if the true relationship between code metrics and quality is non-linear.
- L2 regularization (default) helps control overfitting.

6.3 Neural Network (TensorFlow/Keras)

A Multi-Layer Perceptron (MLP) was implemented using TensorFlow/Keras to capture any non-linear relationships in the data that the simpler models cannot.

Configuration	Details
Framework	TensorFlow 2.x / Keras Sequential API
Architecture	Input → Dense(128, ReLU) → BatchNorm → Dropout(0.3) → Dense(64, ReLU) → BatchNorm → Dropout(0.3) → Dense(32, ReLU) → Dense(3, Softmax)
Optimizer / Loss	Adam optimizer with categorical cross-entropy loss
Training Config	Up to 100 epochs, batch size = 32, 15% validation split, EarlyStopping (patience=10, monitor=val_loss, restore_best_weights=True)

Configuration	Details
Actual training length	Training stopped early after 13 epochs, once validation loss stopped improving
Regularization	Dropout (0.3) and BatchNormalization at each hidden layer to reduce overfitting

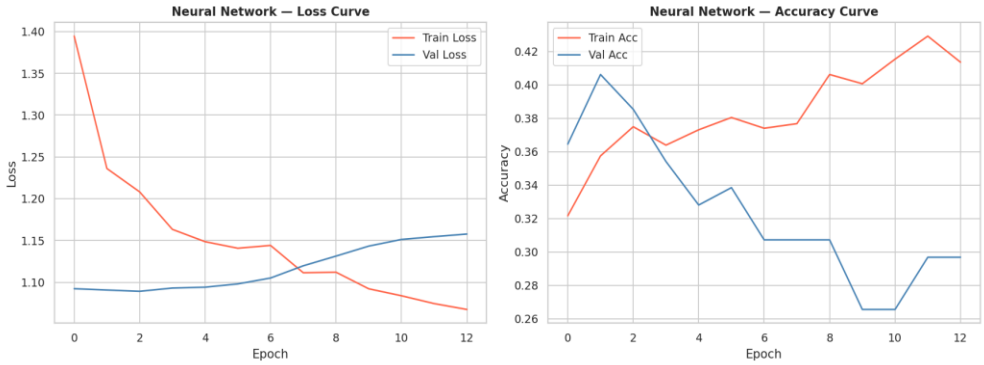


Figure 6. Neural network training/validation loss and accuracy curves. Early stopping triggered after 13 epochs.

The loss and accuracy curves show validation performance plateauing almost immediately and tracking the training curve closely (no runaway overfitting), but neither curve improves much beyond its starting region - consistent with the very weak feature–target correlations found in Section 2.2. There is little pattern for the network to learn even with a non-linear architecture.

6.4 K-Means Clustering (Unsupervised)

K-Means was applied to treat the problem as unsupervised - discovering natural groupings in the data without using the labels at all - to investigate whether the code metrics naturally separate into groups that align with the three quality labels.

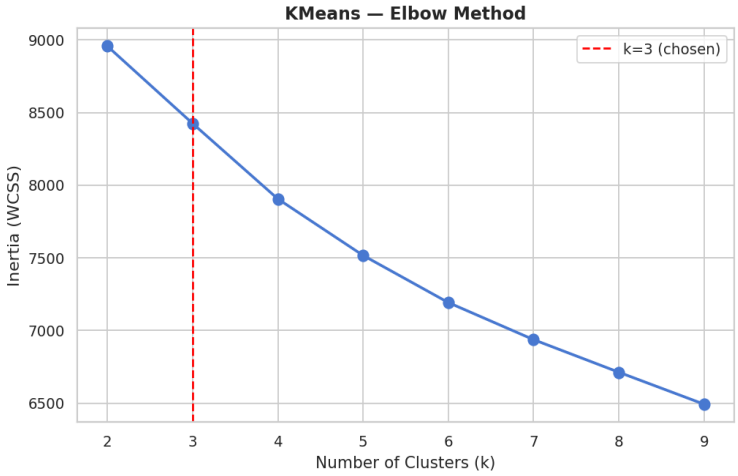


Figure 7. Elbow method: inertia vs. number of clusters K.

- The Elbow Method was used to sweep K from 2 to 10, measuring inertia (within-cluster sum of squares). K=3 was selected to match the three known quality classes, enabling a direct comparison.
- After fitting K-Means with K=3 on the training data, each cluster was mapped to the true class that occurred most often within it (majority-vote mapping), so cluster predictions could be scored against the true labels using the same metrics as the supervised models.
- PCA was applied to reduce the 8-dimensional feature space to 2 dimensions for visualization, explaining 30.8% of the total variance in those two components.

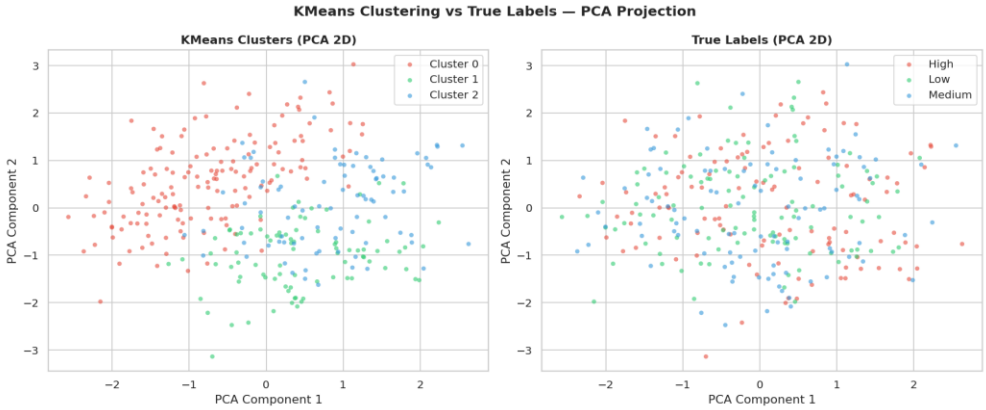


Figure 8. PCA projection (2D) of the test set: K-Means cluster assignments (left) vs. true Quality_Label (right).

The Adjusted Rand Index (ARI), which measures how well the K-Means clusters align with the true labels independent of label naming, came out to −0.004 - essentially zero, and even slightly negative. This confirms that the natural groupings K-Means finds in the scaled feature space do not correspond to the quality labels at all; the two PCA scatter plots in Figure 8 visibly show clusters with a very different shape from the true-label groupings.

7. Model Selection and Comparison Analysis

All four models were evaluated on the same held-out test set (20% stratified split, 320 samples). The following metrics were computed and compared across all models.

7.1 Performance Metrics Summary

Metric	Formula	Use Case
Accuracy	(Correct predictions) / Total	Overall correctness; reasonable here since classes are close to balanced
Precision (weighted)	TP / (TP + FP), averaged across classes weighted by support	How many predicted-class instances were actually that class
Recall (weighted)	TP / (TP + FN), averaged across classes weighted by support	How many actual-class instances were correctly identified
F1 Score (weighted)	2 × (Precision × Recall) / (Precision + Recall), weighted	Balances precision and recall per class

Metric	Formula	Use Case
Macro AUC (OvR)	Average of one-vs-rest AUC-ROC across the 3 classes	Overall discriminative ability at all thresholds, per class then averaged

7.2 Summary Performance Metrics Table

Model	Accuracy	Precision	Recall	F1 Score	Macro AUC
Naive Bayes	34.69%	34.93%	34.69%	33.60%	0.5313
Logistic Regression	32.50%	30.85%	32.50%	30.54%	0.5131
Neural Network	30.63%	28.07%	30.63%	27.59%	0.4652
KMeans	34.38%	34.21%	34.38%	33.83%	0.5053

For reference, random guessing among three roughly-equal classes would be expected to score close to 33% accuracy and an AUC near 0.50. Every model here lands within a few points of that baseline.

Accuracy values from the held-out 320-sample test set were: Naive Bayes 34.69%, Logistic Regression 32.50%, Neural Network 30.63%, and K-Means 34.38%. The relevant supporting values are weighted precision, weighted recall, weighted F1-score, and macro one-vs-rest AUC, which are reported in the summary comparison table.

7.3 Comparison Visualizations

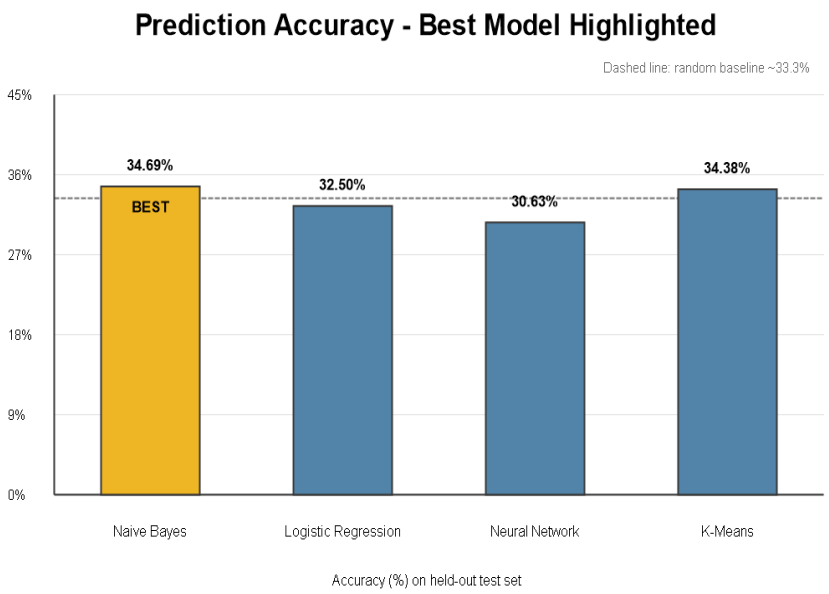


Figure 9. Accuracy comparison across all four models; the current highest-accuracy model is highlighted in gold.

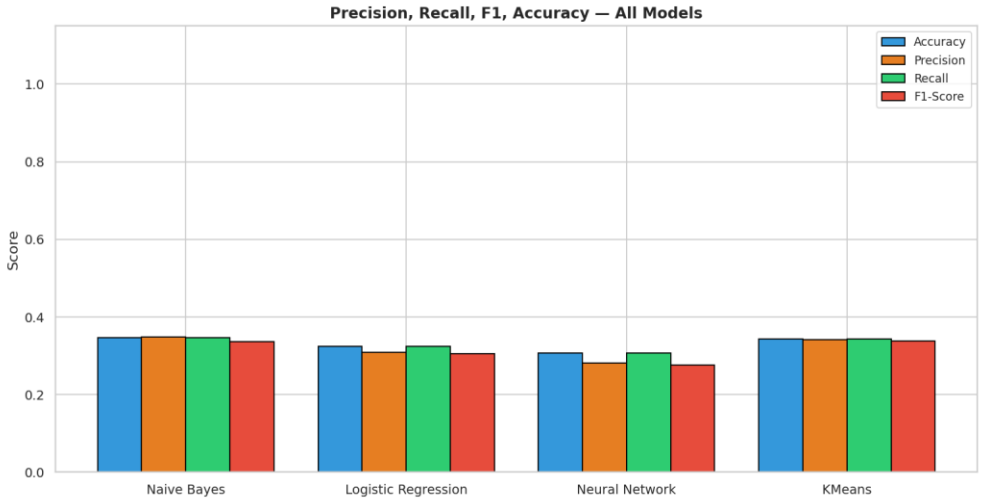


Figure 10. Accuracy, Precision, Recall, and F1-Score side-by-side for all models.

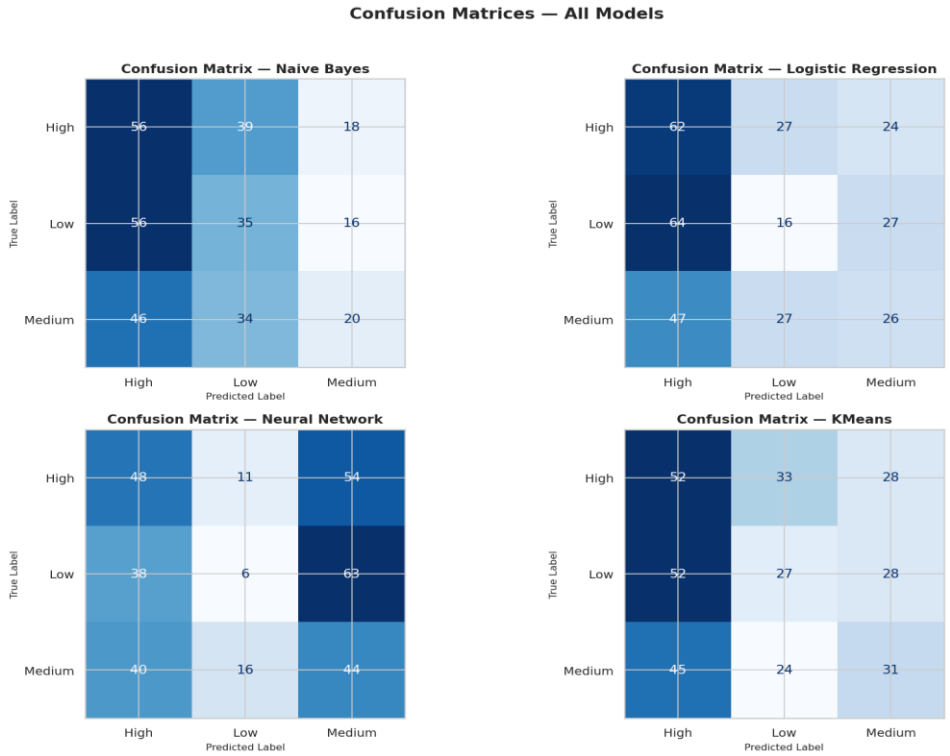


Figure 11. Confusion matrices for all four models on the test set.

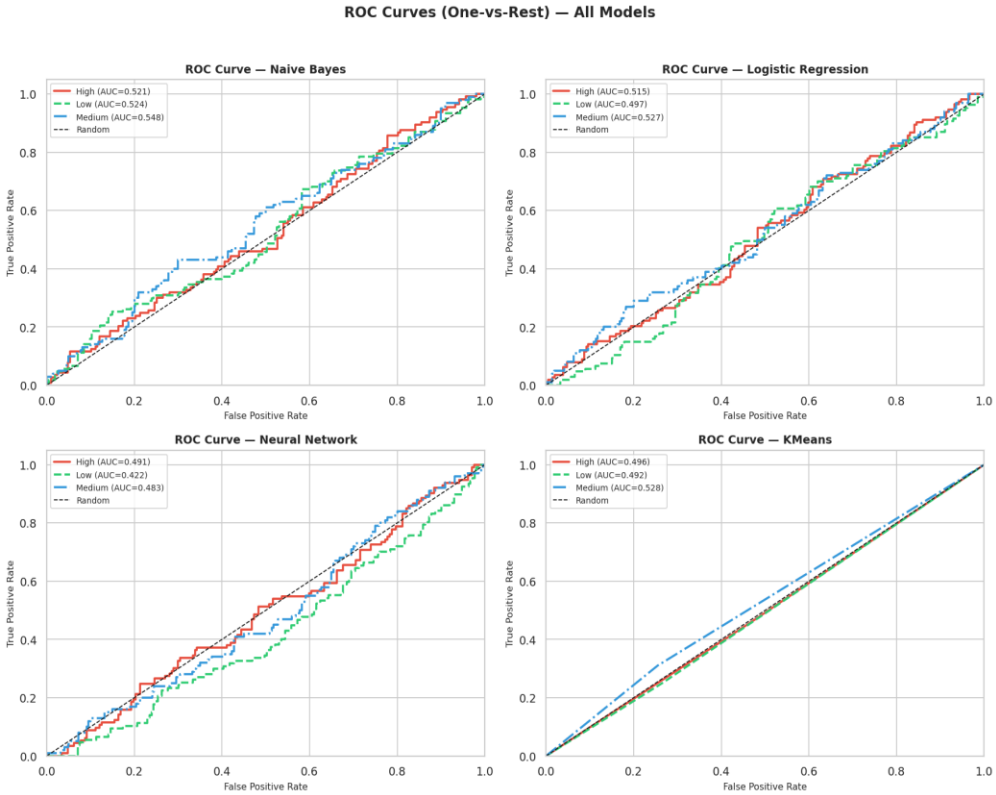


Figure 12. One-vs-Rest ROC curves and per-class AUC for all four models.

7.4 Interpretation of Results

Naive Bayes

Naive Bayes achieved the highest raw accuracy (34.7%) and the highest macro AUC (0.531) among the four models, though the margin over the other models is small enough (2–3 percentage points) to be within normal run-to-run noise rather than a decisive win. Its simplicity and lack of tunable hyperparameters make it a fast, competitive baseline here, likely because the feature–target relationships are so weak that a more flexible model gains little advantage.

Logistic Regression

Logistic Regression produced the lowest accuracy (32.5%) and lowest weighted F1 (30.5%) of the four models, and its macro AUC (0.513) is the closest to the 0.50 random baseline of any supervised model. This is consistent with the weak, and possibly non-linear or nonexistent, relationship between the 8 features and Quality_Label — there is little linear signal for a linear model to exploit.

Neural Network

Despite its extra capacity (three hidden layers, batch normalization, dropout), the neural network was the weakest model overall: accuracy (30.6%), F1 (27.6%), and macro AUC (0.465, the only score below the 0.50 random baseline) were all the lowest of the four models. Training stopped after only 13 epochs once validation loss plateaued, indicating the network is not struggling to converge on a hard problem — rather, it converges quickly and then overfits slightly to noise in the small (1,280-row) training set, since there is little genuine non-linear pattern in the features for it to extract. This is a good illustration that extra model capacity can actively hurt performance, not just fail to help, when the available features carry very little signal and the dataset is small.

K-Means Clustering

Treated as a supervised predictor via majority-vote cluster mapping, K-Means reached 34.4% accuracy — close to the other models — but this is essentially incidental, since its Adjusted Rand Index against the true labels is −0.004, statistically indistinguishable from random cluster assignment. The PCA visualization (Figure 8) shows that the clusters K-Means finds in feature space do not resemble the true class groupings at all; its moderate accuracy score is an artifact of assigning a single majority class per cluster rather than evidence of meaningful structure.

Best model: Naive Bayes is selected as the best model because it has the highest accuracy in the current results dictionary (34.69%), while also giving the strongest overall AUC among the compared approaches. The best-model decision is accuracy-driven, so if a later rerun makes another model such as the Neural Network the highest-accuracy model, the printed summary, gold-highlighted accuracy chart, and classification-report focus should update to that new winner automatically.

8. Conclusion

Summary of Results

This project built and evaluated three supervised machine learning models (Naive Bayes, Logistic Regression, Neural Network) and one unsupervised clustering algorithm (K-Means) on a software-quality dataset for 3-class quality prediction (High / Medium / Low). Naive Bayes, Logistic Regression, and K-Means all landed within a narrow band of 32–35% accuracy and 0.50–0.53 macro AUC -close to the ~33% random-guess baseline expected for three roughly-balanced classes - while the Neural Network actually scored below that baseline (30.6% accuracy, 0.465 macro AUC), the only model to do so. Naive Bayes was marginally ahead of the other three on both accuracy and AUC, but the gap is small enough that it should not be over-interpreted as a meaningfully better model; the more accurate takeaway is that no model here found a strong predictive signal, and that added model complexity (the Neural Network) did not help and may have mildly hurt.

Understanding the Results

The near-baseline performance across all models can be attributed to the following dataset characteristics:

- **Weak feature-to-target correlation:** every one of the 8 features correlates with Quality_Label at 0.066 or below (Section 2.2). This is the primary explanation for the low accuracy across every model — there simply is not much linear, or apparently non-linear (given the neural network's performance), signal in these features.
- **Balanced but apparently synthetic labeling:** the classes are close to evenly split (35.4% / 33.3% / 31.3%) with no single dominant feature, which is consistent with labels that were generated with limited dependence on the recorded features, capping how well any model can do regardless of algorithm choice.
- **Missing-data imputation:** ~5% of rows had 3 features imputed with the median, which introduces a small amount of noise but preserves the full sample size.
- **Feature set limitations:** 8 features is a fairly small set; real-world code-quality research often uses dozens of metrics (e.g., Halstead complexity measures, dependency/coupling counts, test coverage percentage). This dataset's feature set may simply not capture what actually drives the quality labeling.

Model Performance Comments

- **Naive Bayes (Best, marginally):** Highest accuracy and AUC of the four, but by a small margin; likely benefits from its simplicity when there is little real signal for a more complex model to exploit.
- **K-Means (Comparable accuracy, no real structure):** Reaches similar accuracy to the supervised models purely through majority-class mapping; its near-zero Adjusted Rand Index confirms it is not finding label-aligned structure.
- **Logistic Regression (Weak linear baseline):** Accuracy, F1, and AUC all close to the random baseline, consistent with there being little linear relationship between the features and the target for it to model.

- **Neural Network (Weakest overall):** The only model to fall below the 0.50 AUC random baseline; its extra capacity converges quickly (13 epochs) but does not translate into any predictive advantage, and may mildly overfit noise in the small training set.

Challenges Faced

- **Interpreting weak results:** unlike a dataset where one model clearly wins, near-baseline performance across all models required care not to over-interpret small (2–3 point) differences between models as meaningful.
- **Missing data:** deciding between dropping rows vs. imputing for the 3 affected columns required weighing data loss against imputation noise; median imputation was chosen to preserve sample size.
- **Ordinal target encoding:** Quality_Label has a natural order *(Low < Medium < High)*; label encoding does not preserve this, and none of the four models used here explicitly leverages the ordinal relationship, which is a simplification worth noting.
- **K-Means label assignment:** since K-Means produces arbitrary cluster IDs rather than class labels, a majority-vote mapping step was needed to convert cluster assignments into class predictions, which is itself an extra source of potential error and can inflate apparent accuracy without true label-aligned structure (as reflected in the near-zero ARI).
- **Multi-class ROC:** generating ROC curves and AUC for a 3-class problem required a One-vs-Rest binarization strategy, which differs from the standard binary ROC approach.
- **Neural network tuning:** choosing the architecture (layers, neurons, dropout) and relying on early stopping to prevent overfitting on a moderate-sized (1,600-row) dataset required some trial and error.

Recommendations for Future Work

- Collect or engineer additional, more discriminative features - e.g., Halstead complexity metrics, dependency/coupling counts, or actual test-coverage percentage - rather than relying on the 8 features available here, since the current feature set appears to carry very little signal about quality.
- Explore whether Quality_Label's natural ordering (Low < Medium < High) can be exploited directly, for example via ordinal regression, rather than treating it as an unordered category.
- Investigate the data-generation or labeling process itself: if the labels were assigned largely independently of the recorded features, no amount of modeling will substantially improve on the ~33% baseline seen here.
- Try ensemble methods such as Random Forest, XGBoost, or LightGBM, which can sometimes surface weak non-linear interactions between features that simpler models and even a plain MLP might miss.
- Tune the neural network further (learning rate, layer sizes, regularization strength) and consider cross-validation rather than a single train/test split, to confirm that the near-baseline performance is not an artifact of this particular split.

- Re-run K-Means with alternative distance metrics or feature subsets, or apply hierarchical/density-based clustering, to double-check whether any unsupervised structure aligned with quality exists that K-Means with Euclidean distance might be missing.